

Document no.	P0915R0
Date	2018-02-08
Reply-to	Vittorio Romeo < vittorio.romeo@outlook.com (mailto:vittorio.romeo@outlook.com)>, John Lakos < jlakos@bloomberg.net (mailto:jlakos@bloomberg.net)>
Audience	Evolution Working Group
Project	ISO JTC1/SC22/WG21: Programming Language C++

Concept-constrained auto

Abstract

This paper proposes the addition of a *concept-constrained* auto placeholder type for variables. The primary goal is to increase code readability and correctness without sacrificing genericity:

```
template <typename T>
void foo(std::vector<T>& v)
{
    auto<RandomAccessIterator> it{std::begin(v)};
    // ...
}
```

Table of contents

- [Overview](#)
- [Motivation](#)
- [Proposed syntax](#)
- [Wording](#)
- [Future directions](#)
- [Bikeshedding](#)
- [References](#)

Overview

A large part of the Concepts TS ¹ has been merged into the C++20 Standard working draft ². Due to the lack of consensus and also to the existence of reasonable concerns, proposed features such as **placeholders** (*changes to `[dcl.spec.auto]`*) and **abbreviated templates** (*changes to `[dcl.fct]` and `[temp]`*) have not yet been merged into the working draft.

This paper proposes the introduction of a *concept-constrained* `auto` type placeholder, focusing on only the simplest, most common, and most impactful use case, while leaving the door open for future extensions.

The proposed *concept-constrained* `auto` enables users to specify a (constraining) *concept-name* when declaring variables using `auto`:

```
std::list<int> aList;

// Unconstrained `auto`:
auto i0 = std::begin(aList);

// Well-formed usage of concept-constrained `auto`:
auto<BidirectionalIterator> i1 = std::begin(aList);

// Ill-formed usage of concept-constrained `auto`:
auto<RandomAccessIterator> i2 = std::begin(aList); // <== compile-time
    error
```

Motivation

The current *unconstrained* `auto` placeholder works well in many scenarios:

- **When the type is obvious from the *initialization expression*:**

```
auto foo = std::make_shared<Foo>();
```

```
std::map<int, std::string> aMap;
auto it = std::begin(aMap);
```

- **When the type cannot be spelled out explicitly:**

```
auto lambda = []{ /* ... */; };
```

- **Expression templates:**

```
Matrix a{/* ... */}, b{/* ... */}, c{/* ... */};  
auto add = a + b;  
auto mul = add * c;  
consume(mul);
```

In the snippet above, `mul` is not of type `Matrix`; its type encodes the sequence of operations as a compile-time expression tree. Using `Matrix` instead of `auto` could result in performance degradation.

Unfortunately, the inability to constrain `auto` with a concept often results in less-readable code, sometimes with surprising results. Consider the following cases:

- **Invocation of functions in templated contexts:**

```
const auto& myEmployees = getEmployees();  
const auto& senior = mostExperiencedOf(myEmployees);
```

Without reading the signatures (*and possibly even the implementations*) of `getEmployees` and `mostExperiencedOf`, it is unclear what kinds of types `myEmployees` and `senior` might be.

- Is `myEmployees` a concrete container? Is it a lazily evaluated range?
- Is `senior` a reference to an element inside `myEmployees`? Is it an *iterator*?

With the proposed *concept-constrained* `auto`, the code becomes more readable and explicit.

```
const auto<ContiguousContainer>& myEmployees = getEmployees();  
auto<RandomAccessIterator> senior = mostExperiencedOf(myEmployees);
```

More generally, explicit specification of constraining concepts (as in the code snippet above) allows readers to have more refined knowledge about the properties of the types returned by `getEmployees` and `mostExperiencedOf`. The provided (named) constraints make the operations available on the returned values obvious (easily searchable), without sacrificing genericity or performance. Additionally, failure of the exact types to satisfy the requirement would be caught early (*at initialization, rather than at the point of usage*).

- **Making assumptions about types explicit:**

Imagine writing a function that copies the memory of a properly-aligned *standard layout* class to the GPU. Using (unconstrained) `auto` and no *static assertions* could result in undesirable behavior:

```
template <typename Producer>
void uploadToGPU(Producer& producer)
{
    auto item = producer.next();
    gpuMemcpy(dst, &item, sizeof(item)); // <== Potentially UB
}
```

In the code snippet above, `gpuMemcpy` expects `&item` to be a pointer to a *standard-layout* type, but there is nothing enforcing that to be true.

Adding a `static_assert` would prevent mistakes from happening, but might also (for some) detract from the readability of the code:

```
template <typename Producer>
void uploadToGPU(Producer& producer)
{
    auto item = producer.next();
    static_assert(StandardLayoutType<decltype(item)>);
    gpuMemcpy(dst, &item, sizeof(item));
}
```

Using the proposed *concept-constrained* `auto` would achieve a safe and clear result with minimal boilerplate:

```
template <typename Producer>
void uploadToGPU(Producer& producer)
{
    auto<StandardLayoutType> item = producer.next();
    gpuMemcpy(dst, &item, sizeof(item));
}
```

We are confident that proper use of *concept-constrained* auto, where appropriate, will significantly increase the readability and accessibility of Modern C++ programs, especially in large code bases. Note that we are not suggesting that *concept-constrained* auto should always be preferred to (unconstrained) auto -- there are common situations where supplying the optional constraint would be counter indicated:

```
Employee* Company::findFirstSeniorEmployee() const
{
    auto<Iterator> it = std::begin(this->d_employees);
    //      ^~~~~~

    // ...business logic...
    return it == std::end(this->d_employees) ? nullptr
                                           : &*it;
}
```

In the example above, the identifier `it` is already clearly some form of `Iterator`. Moreover, the `Iterator` concept, along with its idiomatic use as being what is expected of the return types for `begin()` and `end()` methods, are familiar to virtually every C++ developer. (We refer to such ubiquitously familiar concepts as **vocabulary concepts**). Making `Iterator` an explicit (named) constraint to `auto` here would be pointless noise – arguably detracting from readability. If, on the other hand, it were the case that (1) the code did not as yet make explicit use of *random-access-iterator* features, and (2) it was foreseeable that such features would soon be required, then constraining the `auto` with `RandomAccessIterator` is exactly what would express that engineering intent:

```

Employee* Company::findFirstSeniorEmployee() const
{
    auto<RandomAccessIterator> it = std::begin(this->d_employees);
    //      ^~~~~~
    //      Required to express engineering intent.

    // ...business logic that does not require a `RandomAccessIterator`
    (yet)...
    return it == std::end(this->d_employees) ? nullptr
                                           : &*it;
}

```

From an engineering standpoint, however, the use of *concept-constrained* `auto` becomes imperative, when dealing with **non-vocabulary** concepts, as this compiler-enforced “documentation” dramatically facilitates developers becoming familiar with previously unseen concepts by name, which – in turn – can be more easily looked up if need be:

```

void Employee::logStatus() const
{
    auto<bdex::OutputStream>& stream = getLogStream();
    //      ^~~~~~

    stream.putString(this->d_name);
    stream.putUint32(this->d_userId);
    stream.putUint8(this->d_statusFlags);
}

```

Developers reading the function above will benefit greatly from the explicit `bdex::OutputStream` concept constraint, especially those who find it unfamiliar:

- Being able to see `bdex::OutputStream` gives readers a name to search for in the code base and its documentation to understand the scope and functionality of the concept.
- The `auto<...>` notation (*as opposed to the “as yet to be adopted” terse notation*) makes it unambiguously clear that `bdex::OutputStream` is a *concept* and **not** a *type*.

If (unconstained) `auto` were used, the reader would need to check the declaration of `getLogStream()` (and possibly its definition) in order to understand what kind of object was being returned.

Proposed syntax

We propose the ability of constraining `auto` with a concept, independently of cv-qualifiers. E.g.

```
auto<Iterator> i0 = foo();
auto<BidirectionalIterator>&& i1 = foo();
const volatile auto<MoveConstructible>& i2 = bar();
```

Wording

The proposed wording copies the terminology used in N4674 in order to make future extensions easier to apply.

[dcl.type.simple]

- Add *constrained-type-specifier* to the grammar for *simple-type-specifiers*.
- Modify paragraph 2 to begin:
 - ~~The *simple-type-specifier* `auto`-specifier is a placeholder for a type to be deduced (10.1.7.4).~~
The *simple-type-specifier* `auto` and *constrained-type-specifiers* are placeholders for a type to be deduced (10.1.7.4).
- Add *constrained-type-specifier* to the table of simple-type-specifiers in Table 11:

Specifier(s)	Type
constrained-type-specifier	placeholder for type to be deduced

[dcl.spec.auto]

- Modify paragraph 1 to begin:
 - ~~The auto and decltype(auto) type-specifiers~~ are used to designate a placeholder type that will be replaced later by deduction from an initializer. The auto and decltype(auto) type-specifiers, and *constrained-type-specifier* are used to designate a placeholder type that will be replaced later by deduction from an initializer.

[dcl.spec.auto.deduct]

- Modify paragraph 3 to begin:
 - ~~If the placeholder is the auto type-specifier, the deduced type T' replacing T is determined using the rules for template argument deduction. If the placeholder is the auto type-specifier or a constrained-type-specifier, the deduced type T' replacing T is determined using the rules for template argument deduction.~~
- Modify paragraph 3:
 - ~~Deduce a value for U using the rules of template argument deduction from a function call (17.8.2.1), where P is a function template parameter type and the corresponding argument is e. If the deduction fails, the declaration is ill-formed.~~ Deduce a value for U using the rules of template argument deduction from a function call (17.8.2.1), where P is a function template parameter type and the corresponding argument is e. If the deduction fails, the declaration is ill-formed. If the used placeholder is a *constrained-type-specifier* with an associated constraint C, the declaration is ill-formed if $C\langle U \rangle$ evaluates to false.

[dcl.spec.auto.constr]

- Add this section to 10.1.7.4.
- Paragraph 1:

- A *constrained-type-specifier* designates a placeholder type and introduces an associated constraint.
 - *constrained-type-specifier*:
 - `auto < qualified-concept-name >`

[Example:

```
template<typename T> concept bool C0 = true;
int f0() { return 42; }
void f1()
{
    auto<C0> i = f0(); // auto<C0> designates a placeholder type
                       // with associated constraint C0
}
```

]

Future directions

This proposal is meant to be as simple as possible but leaves room for future extensions.

- *Concept-constrained* auto placeholders could be extended to work as function parameter types and also as function return types:

```
auto l0 = [](auto<ForwardIterator> x){ /* ... */ };
auto foo() -> auto<MoveConstructible> { /* ... */ };
```

- A terser syntax (such as the one already specified in the concepts TS) that doesn't require `auto<...>` to introduce a type placeholder could be incorporated - the concept name on its own would be enough:

```
ForwardIterator it = foo();
```

...equivalent to...

```
auto<ForwardIterator> it = foo();
```

Note that the proposed syntax is **not** intended to be a temporary solution until consensus on the terse syntax is reached. We believe that *concept-constrained* auto will always be needed even if (when) the terse syntax is voted into the Standard, thanks to its more descriptive and explicit value - especially in public APIs.

Additionally, note that:

```
ForwardIteratorConcept it = foo();
```

and

```
auto<ForwardIterator> it = foo();
```

have roughly the same number of characters. Yet, ForwardIteratorConcept is known to be a *concept* due only to convention, whereas auto<ForwardIterator> implies that ForwardIterator is unambiguously a *concept*. So, regardless of whether or not *terse syntax* will ever make it into the Standard, the feature we have proposed in this paper is **necessary** to allow developers to be unambiguously explicit when referring to **non-vocabulary** concepts.

- Logical composition of concepts could be allowed inside the <...> angle brackets:

```
auto<Erasable && MoveConstructible> = bar();
```

The goal of this proposal is to introduce a simplified yet useful minimal subset of the proposed *placeholder* sections in the Concepts working draft. *Concept-constrained* auto's scope and functionality can then be expanded in the future to reach the power and flexibility of the original design.

Bikeshedding

`<...>` is, in our view, the optimal choice due to its clear, concise, and expressive syntax. Unfortunately, it might not be appropriate as it misleadingly suggests *template instantiation*. Andrew Sutton recommended using the `auto|Concept` syntax instead, which was part of the original proposal for concepts but was dropped in favor of the *terse notation* which we have already addressed. There exist other viable possibilities for the *concept-constrained* `auto` syntax:

- `auto{Concept}`
- `auto:Concept`
- `auto Concept`

We have not considered `auto(Concept)` and `auto[Concept]` because they respectively conflict with *variable initialization* and *structured bindings*.

References

- “A plea for a consistent, terse and intuitive declaration syntax”:
 - <https://groups.google.com/a/isocpp.org/forum/#!topic/std-proposals/yEoi0vEnJ48>
(<https://groups.google.com/a/isocpp.org/forum/#!topic/std-proposals/yEoi0vEnJ48>)
 - https://cor3ntin.github.io/CPPProposals/unified_concept_declaration_syntax/concepts.html
(https://cor3ntin.github.io/CPPProposals/unified_concept_declaration_syntax/concepts.html)
- “Remove abbreviated functions and template-introduction syntax from the Concepts TS”:
 - <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0696r0.html> (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0696r0.html>)
- “Concepts are Adjectives, not Nouns”
 - <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0791r0.pdf> (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0791r0.pdf>)

- “An Adjective Syntax for Concepts”

- <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0807r0.html> (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0807r0.html>)

1. <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/n4674.pdf> (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/n4674.pdf>)↵
2. <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/n4700.pdf> (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/n4700.pdf>)↵